소상공인용 AI 마케팅·고객응대 통합 지원 시스템

④ 시스템 설계서 초안

팀명: 신창섭

팀원: 한재형 · 정회성 · 전지호

작성일: 2026-11-03

버전: v1.0

목차

1. 아키텍처 스타일
   1.1 스타일 선정 및 근거
   1.2 핵심 품질 속성
2. 4+1 관점 아키텍처
   2.1 논리 뷰 (Logical View)
   2.2 프로세스 뷰 (Process View)
   2.3 개발 뷰 (Development View)
   2.4 유스케이스 뷰 (Use Case View)
3. 클래스 다이어그램
   3.1 전체 클래스 관계
   3.2 SOLID 체크리스트
4. 디자인 패턴 적용 설계
   4.1 State 패턴 (예약 상태 관리)
   4.2 Strategy 패턴 (채널별 홍보)
   4.3 Adapter 패턴 (AI API 연동)
5. ERD 설계
   5.1 엔티티 목록 및 속성
   5.2 관계 정의
6. 화면 흐름도
7. 프로젝트 골격

1. 아키텍처 스타일

1.1 스타일 선정 및 근거

선정 스타일: 3 계층 레이어드 아키텍처 (Layered Architecture)

계층	구성 요소	역할
Presentation	main.py, app/presentation/	CLI 메뉴 출력, 사용자 입력 처리
Service	app/services/	비즈니스 로직, 디자인 패턴 호출
Repository	app/repositories/	DB CRUD 추상화
Model/DB	app/models/, SQLite	SQLAlchemy ORM, 데이터 저장

선정 근거

- 관심사 분리(Separation of Concerns): 각 계층이 단일 책임을 가져 유지보수 용이
- 팀 역할 분담에 최적: 한재형(F-01), 정회성(F-02·F-03), 전지호(F-04·F-05) 독립 개발 가능
- 테스트 용이성: Repository 계층을 Mock 으로 교체하여 단위 테스트 가능
- 확장성: Adapter 패턴으로 AI API 교체 가능, Strategy 패턴으로 채널 추가 용이
- 소규모 프로젝트 적합: Microservice 대비 구현 복잡도 낮고 팀 학습 곡선 완만

계층 간 의존 규칙

Presentation → Service → Repository → Model (단방향, 역방향 import 금지)

1.2 핵심 품질 속성

품질 속성	목표	측정 지표	검증 방법
유지보수성 (Maintainability)	기능 추가·수정 시 변경 범위를 1 개 계층 이내로 제한	변경 영향 클래스 수 ⩽ 3 개	SOLID 위반 리뷰, 코드 리뷰 체크리스트
확장성 (Extensibility)	새 AI API 또는 홍보 채널 추가 시 기존 코드 무수정	OCP 준수율 100%	Adapter/Strategy 패턴 적용 후 신규 클래스만 추가 확인
신뢰성 (Reliability)	AI API 오류 시 시스템 전체 중단 없이 대체 응답 제공	API 오류 시 fallback 성공률 100%	오류 주입 테스트(Mock API 실패 시 처리 확인)

2. 4+1 관점 아키텍처

Kruchten(1995)의 4+1 관점 모델을 적용하여 시스템을 다각도로 기술한다.

2.1 논리 뷰 (Logical View)

시스템의 주요 클래스와 계층 구조를 표현한다.

[Presentation Layer]
MainMenu → StoreMenu, ReservationMenu, InquiryMenu, MarketingMenu, ChannelMenu

[Service Layer]
StoreService, MenuService, FAQService
ReservationService (State 패턴 사용)
InquiryService (Adapter 패턴 사용)
MarketingService, ChannelService (Strategy 패턴 사용)

[Repository Layer]
StoreRepository, MenuRepository, FAQRepository
ReservationRepository, CustomerRepository
InquiryRepository

[Model/DB Layer]
Store, Menu, FAQ, Reservation, Customer, Inquiry (SQLAlchemy ORM)
SQLite DB (sw_make.db)

[Pattern Classes]
State: ReservationState, PendingState, ConfirmedState, CompletedState, CancelledState
Strategy: ChannelStrategy, SNSStrategy, BlogStrategy, KakaoStrategy
Adapter: AIAdapter, OpenAIAdapter, ClaudeAdapter

2.2 프로세스 뷰 (Process View)

런타임 프로세스와 주요 시나리오별 흐름을 기술한다.

시나리오 1: AI 고객 문의 응대 (F-03)

- 고객이 CLI 에 질문 입력
- InquiryMenu → InquiryService.respond(question)
- InquiryService → AIAdapter.ask(question) 호출
- AIAdapter → OpenAI/Claude API HTTP 요청
- AI 응답 수신 → DB 의 FAQ/메뉴 정보와 대조

- 불일치 시 경고 + DB 기준 정보 병기하여 출력

시나리오 2: 예약 상태 전이 (F-02)

- 점주가 예약 목록에서 예약 선택
- ReservationMenu → ReservationService.confirm(id)
- ReservationService → Reservation.state.confirm() 호출
- PendingState → ConfirmedState 전이 후 DB 저장

시나리오 3: 마케팅 콘텐츠 생성 (F-04·F-05)

- 점주가 이벤트 정보 입력 (메뉴명, 할인율)
- MarketingService → AIAdapter.generate(prompt)
- AI 홍보 문구 초안 수신
- 채널 선택 → ChannelStrategy.format(content)
- 글자수·이모지·해시태그 규칙 검사 후 최종 출력

2.3 개발 뷰 (Development View)

소스 코드의 모듈 구조와 패키지 의존성을 기술한다.

패키지	주요 모듈	담당자
app/models/	store.py, menu.py, faq.py, reservation.py, customer.py, inquiry.py	전지호
app/repositories/	store_repo.py, menu_repo.py, faq_repo.py, reservation_repo.py, customer_repo.py	한재형
app/services/	store_service.py, reservation_service.py, inquiry_service.py, marketing_service.py	한재형·정회성
app/patterns/state/	reservation_state.py, pending.py, confirmed.py, completed.py, cancelled.py	정회성
app/patterns/strategy/	channel_strategy.py, sns_strategy.py, blog_strategy.py, kakao_strategy.py	전지호
app/patterns/adapter/	ai_adapter.py, openai_adapter.py, claude_adapter.py	정회성
app/presentation/	main_menu.py, store_menu.py, reservation_menu.py, inquiry_menu.py, marketing_menu.py	한재형

기술 스택

- 언어: Python 3.11+
- ORM: SQLAlchemy 2.x
- 유효성 검증: Pydantic v2
- DB: SQLite (sw_make.db)
- AI API: OpenAI gpt-4o-mini / Anthropic Claude API
- 테스트: pytest + pytest-mock

2.4 유스케이스 뷰 (Use Case View)

시스템의 주요 행위자와 유스케이스를 기술한다.

행위자	주요 유스케이스	관련 기능
점주	UC-01~05 매장·메뉴·FAQ 관리 UC-07~10 예약 관리 UC-14~19 마케팅·채널·대시보드	F-01, F-02, F-04, F-05
고객	UC-06 예약 요청 UC-11 문의 질문 입력	F-02, F-03
AI API	UC-12 AI 답변 생성 UC-15 AI 홍보 문구 생성	F-03, F-04

※ 유스케이스 전체 목록(UC-01~UC-19)은 docs/usecase.md 참조

3. 클래스 다이어그램

3.1 전체 클래스 관계

클래스	계층	주요 속성	주요 메서드	관계
Store	Model	id, name, address, phone, hours	-	has-many Menu, FAQ
Menu	Model	id, store_id, name, price, description	-	belongs-to Store
FAQ	Model	id, store_id, question, answer	-	belongs-to Store
Customer	Model	id, name, phone, visit_count	-	has-many Reservation
Reservation	Model	id, customer_id, datetime, party_size, state	-	belongs-to Customer, uses ReservationState
Inquiry	Model	id, question, ai_answer, db_verified	-	-
StoreService	Service	-	create(), update(), get()	uses StoreRepository
ReservationService	Service	-	create(), confirm(), complete(), cancel()	uses ReservationRepository, State 패턴
InquiryService	Service	-	respond(question)	uses AIAdapter, InquiryRepository
MarketingService	Service	-	generate(event_info)	uses AIAdapter
ChannelService	Service	-	format_and_check(content, channel)	uses ChannelStrategy
AIAdapter	Adapter	api_key	ask(prompt), generate(prompt)	<<interface>>
OpenAIAdapter	Adapter	model	ask(), generate()	implements AIAdapter
ClaudeAdapter	Adapter	model	ask(), generate()	implements AIAdapter
ChannelStrategy	Strategy	-	format(content)	<<interface>>
SNSStrategy	Strategy	max_chars=500	format()	implements ChannelStrategy
BlogStrategy	Strategy	max_chars=2000	format()	implements ChannelStrategy
KakaoStrategy	Strategy	max_chars=1000	format()	implements ChannelStrategy

ReservationState	State	-	confirm(), complete(), cancel()	<<interface>>
PendingState	State	-	confirm(), cancel()	implements ReservationState
ConfirmedState	State	-	complete(), cancel()	implements ReservationState
CompletedState	State	-	(전이 불가)	implements ReservationState
CancelledState	State	-	(전이 불가)	implements ReservationState

3.2 SOLID 체크리스트

원칙	내용	적용 방법	준수 여부
SRP (단일 책임)	클래스는 하나의 변경 이유만 가짐	Service/Repository/Model 계층 분리, 각 클래스 20 줄 이하 권장	✅
OCP (개방-폐쇄)	확장에 열려 있고 수정에 닫혀 있음	AIAdapter·ChannelStrategy 인터페이스로 신규 구현체 추가만으로 확장	✅
LSP (리스코프 치환)	하위 타입은 상위 타입 대체 가능	OpenAIAdapter/ClaudeAdapter 가 AIAdapter 인터페이스 완전 구현	✅
ISP (인터페이스 분리)	클라이언트별 최소 인터페이스	AIAdapter 와 ChannelStrategy 를 별도 인터페이스로 분리	✅
DIP (의존성 역전)	고수준 모듈이 저수준에 의존하지 않음	InquiryService 는 AIAdapter 추상에 의존, 구체 구현체는 주입	✅

4. 디자인 패턴 적용 설계

4.1 State 패턴 — 예약 상태 관리 (F-02)

적용 목적

예약 상태(대기→확정→완료/취소)에 따라 허용 동작이 달라지는 복잡한 조건 분기를 제거하고, 각 상태를 독립 클래스로 캡슐화한다.

상태 전이

생성 → 대기(Pending) → 확정(Confirmed) → 완료(Completed)

↘️ 취소(Cancelled) ← (대기 또는 확정에서 가능)

상태 클래스	허용 메서드	금지 메서드	전이 대상
PendingState	confirm(), cancel()	complete()	ConfirmedState, CancelledState
ConfirmedState	complete(), cancel()	confirm()	CompletedState, CancelledState
CompletedState	없음	confirm(), complete(), cancel()	전이 불가
CancelledState	없음	confirm(), complete(), cancel()	전이 불가

적용 경로: app/patterns/state/

4.2 Strategy 패턴 — 채널별 홍보 (F-05)

적용 목적

SNS·블로그·카카오톡 채널별로 포맷 규칙(글자수, 이모지, 해시태그)이 다른 홍보 문구 생성을 전략 객체로 분리하여 OCP 를 준수한다.

전략 클래스	채널	글자수 제한	이모지	해시태그
SNSStrategy	Instagram / Twitter	500 자 이하	허용 (권장)	5 개 이하
BlogStrategy	네이버 블로그	2000 자 이하	선택	제한 없음
KakaoStrategy	카카오톡 채널	1000 자 이하	허용	사용 안 함

적용 경로: app/patterns/strategy/

4.3 Adapter 패턴 — AI API 연동 (F-03·F-04)

적용 목적

OpenAI 와 Claude API 의 인터페이스 차이를 Adapter 로 추상화하여, Service 계층이 특정 AI API 에 종속되지 않도록 한다. 향후 다른 AI API 추가 시 새 Adapter 클래스만 작성하면 된다.

인터페이스	메서드	반환 타입	설명
AIAdapter (추상)	ask(prompt: str) -> str	str	고객 문의 응답 생성
AIAdapter (추상)	generate(prompt: str) -> str	str	마케팅 문구 생성
OpenAIAdapter	ask(), generate()	str	openai.ChatCompletion 호출
ClaudeAdapter	ask(), generate()	str	anthropic.Anthropic 호출

적용 경로: app/patterns/adapter/

5. ERD 설계

5.1 엔티티 목록 및 속성

엔티티	속성 (PK 굵게)	설명
Store	id(PK), name, address, phone, business_hours, created_at	매장 정보
Menu	id(PK), store_id(FK), name, price, description, is_active, created_at	메뉴 정보
FAQ	id(PK), store_id(FK), question, answer, created_at	자주 묻는 질문
Customer	id(PK), name, phone(UNIQUE), visit_count, created_at	고객 정보
Reservation	id(PK), customer_id(FK), datetime, party_size, status, cancel_reason, created_at	예약 정보
Inquiry	id(PK), question, ai_answer, is_verified, db_note, created_at	AI 문의 응답 이력
MarketingLog	id(PK), event_name, channel, content, created_at	마케팅 콘텐츠 이력

5.2 관계 정의

관계	카디널리티	외래키	설명
Store → Menu	1 : N	Menu.store_id	한 매장은 여러 메뉴를 가진다
Store → FAQ	1 : N	FAQ.store_id	한 매장은 여러 FAQ 를 가진다
Customer → Reservation	1 : N	Reservation.customer_id	한 고객은 여러 예약을 가진다
Reservation → (State)	1 : 1	Reservation.status (ENUM)	예약은 하나의 상태를 가진다

ERD 텍스트 표현

Store ||--o{ Menu : "has"

Store ||--o{ FAQ : "has"

Customer ||--o{ Reservation : "makes"

Reservation }o--|| ReservationStatus : "has_state"

6. 화면 흐름도

CLI 메인 메뉴 구조 및 각 기능별 화면 전환 흐름을 기술한다.

메인 메뉴 구조

[시작] → 메인 메뉴

1. 매장·메뉴·FAQ 관리 (F-01)
2. 예약·고객 관리 (F-02)
3. AI 고객 문의 응대 (F-03)
4. 마케팅 콘텐츠 생성 (F-04)
5. 채널별 홍보·대시보드 (F-05)
0. 종료

메뉴	화면 흐름	입력 → 출력
1. 매장 관리	메인→매장메뉴→(등록/수정/메뉴추가/FAQ 등록/FAQ 검색)	매장명·주소·전화·영업시간 입력 → "저장 완료" 출력
2. 예약 관리	메인→예약메뉴→(예약목록/상태변경/고객이력)	예약 선택 → 상태 전이 결과 출력
3. AI 문의	메인→문의입력→AI 응답→DB 검증결과	질문 입력 → AI 답변+DB 대조결과 출력, 불일치 시 ⚠ 경고
4. 마케팅	메인→이벤트정보입력→AI 문구생성→채널선택→최종출력	이벤트명·메뉴·할인율 입력 → 채널별 포맷 문구 출력
5. 채널·대시보드	메인→채널선택→포맷규칙검사→출력 / 대시보드조회	채널 선택 → 규칙 검사 결과 + 당일 예약·문의 현황 출력

7. 프로젝트 골격

실제 구현에서 생성될 전체 디렉터리 및 파일 구조이다.

```
sw_make/
├── main.py                    # CLI 진입점
├── requirements.txt           # 의존성 목록
├── CLAUDE.md                  # AI 협업 규칙
├── sw_make.db                 # SQLite DB (런타임 생성)
├── app/
│   ├── __init__.py
│   ├── models/                # SQLAlchemy ORM 모델
│   │   ├── __init__.py
│   │   ├── store.py           # Store, Menu, FAQ
│   │   ├── reservation.py     # Reservation, Customer
│   │   └── inquiry.py         # Inquiry, MarketingLog
│   ├── repositories/          # DB CRUD 추상화
│   │   ├── __init__.py
│   │   ├── store_repo.py
│   │   ├── menu_repo.py
│   │   ├── faq_repo.py
│   │   ├── reservation_repo.py
│   │   └── customer_repo.py
│   ├── services/              # 비즈니스 로직
│   │   ├── __init__.py
│   │   ├── store_service.py       # F-01
│   │   ├── reservation_service.py # F-02
│   │   ├── inquiry_service.py     # F-03
│   │   ├── marketing_service.py   # F-04
│   │   └── channel_service.py     # F-05
│   ├── patterns/
│   │   ├── state/             # State 패턴 (예약 상태)
│   │   │   ├── reservation_state.py   # 추상 인터페이스
│   │   │   ├── pending_state.py
│   │   │   ├── confirmed_state.py
│   │   │   ├── completed_state.py
│   │   │   └── cancelled_state.py
│   │   ├── strategy/          # Strategy 패턴 (채널별 홍보)
│   │   │   ├── channel_strategy.py    # 추상 인터페이스
```

```
│   │   │   ├── sns_strategy.py
│   │   │   ├── blog_strategy.py
│   │   │   └── kakao_strategy.py
│   │   └── adapter/              # Adapter 패턴 (AI API)
│   │       ├── ai_adapter.py         # 추상 인터페이스
│   │       ├── openai_adapter.py
│   │       └── claude_adapter.py
│   ├── presentation/             # CLI 메뉴 (입출력)
│   │   ├── __init__.py
│   │   ├── main_menu.py
│   │   ├── store_menu.py         # F-01
│   │   ├── reservation_menu.py   # F-02
│   │   ├── inquiry_menu.py       # F-03
│   │   ├── marketing_menu.py     # F-04
│   │   └── channel_menu.py       # F-05
│   └── utils/
│       ├── database.py           # DB 연결, 세션 관리
│       └── config.py             # 환경변수, API 키
├── docs/                         # 설계 문서
│   ├── usecase.md
│   ├── design/                   # 설계 산출물 DOCX·PDF
│   └── README.md
└── tests/                        # 단위·통합 테스트
    ├── __init__.py
    ├── test_store_service.py
    ├── test_reservation_service.py
    ├── test_inquiry_service.py
    └── test_channel_service.py
```